

API Documentation

This document lists every API endpoint referenced in the frontend file `frontend/indoor-analysis.js`.

Base Server

- Analysis APIs are called on `http://localhost:3000`
- Some design storage APIs are written as relative routes such as `/api/save-design`
- In the current frontend code, design storage is still handled in `localStorage` and those backend routes are marked as future backend APIs

1. POST `/api/indoor-analyze`

Purpose

Analyzes a room layout created in Design Mode. The frontend sends the selected room type and all placed objects in the room. The backend is expected to return feng shui scoring, factor-by-factor explanations, and five-element balance data.

Full URL Used by Frontend

`http://localhost:3000/api/indoor-analyze`

Method

POST

Headers

Content-Type: application/json

Request Body

```
{
  "roomType": "bedroom",
  "elements": [
    {
      "type": "bed",
      "position": {
        "x": 1.5,
        "y": 0,
        "z": -2.2
      },
      "fengShui": {
        "element": "earth",
```

```

        "energy": "yin",
        "placement": "important",
        "bagua": "relationship"
    }
},
{
    "type": "desk",
    "position": {
        "x": -1,
        "y": 0,
        "z": 1.8
    },
    "fengShui": {
        "element": "wood",
        "energy": "yang",
        "placement": "power",
        "bagua": "career"
    }
}
]
}

```

Request Field Explanation

roomType Room category selected by the user. The code falls back to **bedroom** if no value is selected.

elements Array of all objects placed in the design canvas.

elements[].type Object type placed in the room, such as bed, sofa, desk, table, chair, wardrobe, bookshelf, tv, mirror, painting, clock, vase, rug, curtain, window, door, fountain, crystals, bamboo, plant, bonsai, flowers, lamp, chandelier, or candle.

elements[].position 3D placement coordinates of the object in the scene.

elements[].position.x Horizontal position in the room.

elements[].position.y Vertical position in the room.

elements[].position.z Depth position in the room.

elements[].fengShui Feng shui metadata attached to the object.

elements[].fengShui.element Five-element type associated with the object, such as wood, fire, earth, metal, or water.

elements[].fengShui.energy Energy category, typically yin, yang, or neutral.

`elements[].fengShui.placement` Placement meaning used by the analysis logic, such as `important`, `center`, `supportive`, `power`, `decorative`, or `entry`.

`elements[].fengShui.bagua` Bagua area association, such as `relationship`, `family`, `career`, `health`, `support`, `wealth`, `knowledge`, `fame`, `creativity`, or `peace`.

Expected Success Response

The frontend expects the API to return a wrapper object with `success` and `data`.

```
{
  "success": true,
  "data": {
    "scores": {
      "overall": 84,
      "element_balance": 79,
      "energy_balance": 81,
      "space_flow": 76,
      "functional_layout": 85,
      "wood": 70,
      "fire": 62,
      "earth": 88,
      "metal": 57,
      "water": 68
    },
    "factors": [
      {
        "title": "Bed Placement",
        "score": 82,
        "confidence": "High",
        "dataSources": [
          "Room Layout",
          "Command Position Rules"
        ],
        "mainIssue": "The bed is mostly stable but is too close to the circulation path.",
        "current": [
          "Bed has wall support",
          "Bed orientation is generally calming"
        ],
        "improve": [
          "Increase distance from the main walkway",
          "Strengthen support on both sides"
        ]
      }
    ]
  }
}
```

```

        "title": "Energy Flow",
        "score": 75,
        "confidence": "Medium",
        "dataSources": [
            "Object Distribution",
            "Qi Flow Heuristics"
        ],
        "mainIssue": "Qi flow is acceptable but some furniture creates minor blockage.",
        "current": [
            "Main room center remains relatively open"
        ],
        "improve": [
            "Reduce clutter near the door",
            "Open the path between key furniture zones"
        ]
    }
],
"fiveElements": {
    "wood": 70,
    "fire": 62,
    "earth": 88,
    "metal": 57,
    "water": 68
},
"summary": "The room shows good grounding and function, with some improvement needed in",
"suggestions": [
    "Move the desk slightly away from the bed",
    "Introduce one metal element to improve balance"
]
}
}

```

Response Field Explanation

`success` Boolean flag checked by the frontend before rendering results.

`data` Main analysis payload.

`data.scores` Category score object displayed in the score dashboard.

`data.scores.overall` Overall feng shui score shown in the main gauge.

`data.scores.element_balance` Score for distribution of the five elements.

`data.scores.energy_balance` Score for yin-yang balance.

`data.scores.space_flow` Score for movement and qi circulation.

`data.scores.functional_layout` Score for practical placement and usability

of the room.

`data.scores.wood`, `data.scores.fire`, `data.scores.earth`, `data.scores.metal`, `data.scores.water` Numeric five-element scores shown in the breakdown and radar chart.

`data.factors` Array of explanation cards shown in the UI.

`data.factors[] .title` Name of the factor being analyzed.

`data.factors[] .score` Numeric score for that factor.

`data.factors[] .confidence` Confidence label displayed by the frontend. If missing, the UI falls back to `Medium`.

`data.factors[] .dataSources` Optional array of evidence sources used by the analysis.

`data.factors[] .mainIssue` Short explanation of the main feng shui observation for the factor.

`data.factors[] .current` Array of positive observations already working well.

`data.factors[] .improve` Array of corrections or improvements recommended.

`data.fiveElements` Optional object for radar-chart rendering. If it is missing, the frontend falls back to the five elemental values inside `data.scores`.

`data.summary` Short overall summary of the room analysis.

`data.suggestions` Optional list of general design recommendations.

Error Handling Expected by Frontend

If the HTTP status is not successful, the frontend reads the response as plain text and throws an error.

If the response is JSON but `success` is `false`, the frontend expects:

```
{
  "success": false,
  "error": "Analysis failed because the room data is incomplete"
}
```

Frontend Behavior

- The request is sent only if at least one design element has been placed.
- The Analyze button is disabled during the request.
- On success, the UI renders score cards, factor cards, a radar chart, and chatbot data.
- On failure, the frontend shows an alert asking the user to check whether the backend server is running on port 3000.

2. POST /api/indoor-photo-analyze

Purpose

Analyzes uploaded or camera-captured room photos. The frontend requires at least 3 photos. If the guided camera mode is used, it recommends all 5 directional photos for best accuracy.

Full URL Used by Frontend

`http://localhost:3000/api/indoor-photo-analyze`

Method

POST

Headers

Content-Type: application/json

Request Body

```
{
  "roomType": "general",
  "source": "camera_guided",
  "photos": {
    "north": "...",
    "south": "...",
    "east": "...",
    "west": "...",
    "floor": "..."
  }
}
```

Request Field Explanation

roomType Currently always sent as **general** by the frontend.

source Indicates how the images were obtained.

Possible values:

- **camera_guided** when the user captures directional photos from the guided camera flow
- **camera_video** when frames are extracted from a recorded room video
- **photo_upload** when photos are manually uploaded

photos Object containing up to 5 image entries.

photos.north Base64 or data URL image for the north-facing view.

photos.south Base64 or data URL image for the south-facing view.

photos.east Base64 or data URL image for the east-facing view.

photos.west Base64 or data URL image for the west-facing view.

photos.floor Base64 or data URL image for the floor view.

Expected Success Response

The frontend expects the response format below.

```
{
  "success": true,
  "data": {
    "overallScore": 81,
    "categories": {
      "lighting": 84,
      "spaceFlow": 76,
      "colorHarmony": 79,
      "furniture": 82,
      "declutter": 74
    },
    "fiveElements": {
      "wood": 66,
      "fire": 58,
      "earth": 80,
      "metal": 61,
      "water": 69
    },
    "recommendations": [
      "Improve lighting in the north corner",
      "Reduce clutter near the main circulation path",
      "Add a balanced wood element for growth energy"
    ],
    "meta": {
      "roomType": "general",
      "photosAnalyzed": 5,
      "source": "camera_guided"
    }
  }
}
```

Response Field Explanation

success Boolean success flag checked before rendering results.

data Main analysis object for photo results.

`data.overallScore` Overall indoor feng shui score.

`data.categories` Category scores shown in the upload results UI.

`data.categories.lighting` Score for natural and artificial lighting quality.

`data.categories.spaceFlow` Score for movement, openness, and qi circulation.

`data.categories.colorHarmony` Score for how well room colors align and support feng shui balance.

`data.categories.furniture` Score for furniture placement quality.

`data.categories.declutter` Score for clutter control and visual cleanliness.

`data.fiveElements` Optional five-elements breakdown. The frontend uses it for chatbot data and supporting interpretation if provided.

`data.recommendations` List of recommended improvements.

`data.meta.roomType` Room type used for the analysis.

`data.meta.photosAnalyzed` Number of photos successfully processed.

`data.meta.source` Source label indicating upload mode.

Error Handling Expected by Frontend

The frontend parses the response as JSON and treats the request as failed when either of these is true:

- the HTTP status is not OK
- `success` is `false`

Example error format:

```
{
  "success": false,
  "error": "At least 3 room photos are required for analysis"
}
```

Frontend Validation Before Request

- Minimum 3 photos are required.
- If guided camera capture is used and fewer than 5 directional photos exist, the user gets a confirmation warning before continuing.
- The button text changes to **Analyzing Photos...** during the request.

3. POST /api/save-design

Status

Planned backend API. Present in frontend comments only. Not actively called in the current implementation.

Purpose

Saves a user-created design to persistent backend storage.

Route Used in Commented Code

/api/save-design

Method

POST

Headers

Content-Type: application/json

Authorization: Bearer <token>

Request Body Built by Frontend

```
{
  "name": "My Bedroom Layout",
  "placedElements": [
    {
      "type": "bed",
      "position": {
        "x": 1.5,
        "y": 0,
        "z": -2.2
      },
      "fengShui": {
        "element": "earth",
        "energy": "yin",
        "placement": "important",
        "bagua": "relationship"
      }
    }
  ],
  "objects": [
    {
      "type": "bed",
      "position": {
        "x": 1.5,
        "y": 0,
        "z": -2.2
      },
      "rotation": 1.57,
      "fengShui": {
```

```

        "element": "earth",
        "energy": "yin",
        "placement": "important",
        "bagua": "relationship"
    }
}
],
"roomType": "bedroom",
"timestamp": "2026-04-04T10:30:00.000Z"
}

```

Request Field Explanation

name Design name entered by the user or auto-generated like `#design1`.

placedElements Simplified list of placed items used in the editor state.

objects Detailed object data from the 3D interaction manager, including position and rotation used to recreate the scene.

roomType Current selected room type.

timestamp ISO timestamp showing when the design was saved.

Expected Success Response

The current frontend comment only checks `response.ok` and then reads JSON. A practical success response would be:

```

{
  "success": true,
  "message": "Design saved successfully",
  "designId": "1712220000000"
}

```

Notes

- In the current implementation, save behavior uses `localStorage` instead of backend storage.
- If this backend route is implemented later, it should preserve the same field names to stay compatible with the frontend.

4. GET `/api/get-designs`

Status

Planned backend API. Present in frontend comments only. Not actively called in the current implementation.

Purpose

Returns the list of previously saved designs for the current authenticated user.

Route Used in Commented Code

/api/get-designs

Method

GET

Headers

Authorization: Bearer <token>

Expected Success Response

The frontend expects an array of saved design objects.

```
[
  {
    "id": "1712220000000",
    "name": "Living Room Layout",
    "roomType": "living_room",
    "timestamp": "2026-04-04T10:30:00.000Z",
    "objects": [
      {
        "type": "sofa",
        "position": {
          "x": 0,
          "y": 0,
          "z": -1
        },
        "rotation": 0,
        "fengShui": {
          "element": "earth",
          "energy": "yin",
          "placement": "center",
          "bagua": "family"
        }
      }
    ]
  }
]
```

Response Field Explanation

id Unique design identifier.

name Display name shown in the history modal.

roomType Saved room type.

timestamp Date and time used for sorting and display.

objects Saved 3D objects used to reconstruct the layout during editing.

Notes

- The UI sorts designs by **timestamp** in descending order.
- The UI displays name, room, object count, and time.

5. GET /api/get-design/:designId

Status

Planned backend API. Present in frontend comments only. Not actively called in the current implementation.

Purpose

Fetches one saved design so the user can reopen and edit it.

Route Used in Commented Code

/api/get-design/{designId}

Method

GET

Headers

Authorization: Bearer <token>

Path Parameter

designId Unique ID of the saved design to load.

Expected Success Response

```
{  
  "id": "1712220000000",  
  "name": "Bedroom Layout",  
  "roomType": "bedroom",  
  "timestamp": "2026-04-04T10:30:00.000Z",  
}
```

```

"objects": [
  {
    "type": "bed",
    "position": {
      "x": 1.5,
      "y": 0,
      "z": -2.2
    },
    "rotation": 1.57,
    "fengShui": {
      "element": "earth",
      "energy": "yin",
      "placement": "important",
      "bagua": "relationship"
    }
  }
]
}

```

Response Field Explanation

`id` Design identifier.

`name` Design name shown in the editor and history modal.

`roomType` Restored into the room type selector.

`timestamp` Used for display and saved history context.

`objects` Array used by the frontend to rebuild the 3D scene.

`objects[].type` Object type to recreate.

`objects[].position` Saved position for the object.

`objects[].rotation` Saved Y-axis rotation applied during restore.

`objects[].fengShui` Feng shui metadata associated with the object.

Notes

- If unsaved changes exist, the frontend asks for confirmation before loading the design.
- If the design is not found, the UI shows a `Design not found` error.

6. DELETE `/api/delete-design/:designId`

Status

Planned backend API. Present in frontend comments only. Not actively called in the current implementation.

Purpose

Deletes a saved design from backend storage.

Route Used in Commented Code

`/api/delete-design/{designId}`

Method

DELETE

Headers

Authorization: Bearer <token>

Path Parameter

designId Unique design ID to delete.

Expected Success Response

The current frontend only checks `response.ok`. A practical backend response would be:

```
{
  "success": true,
  "message": "Design deleted successfully"
}
```

Notes

- In the current implementation, deletion is handled with `localStorage`.
- After deletion, the frontend refreshes the history modal.
- If the deleted design is the current editing design, the editing state is cleared.

Summary of APIs Used in This Frontend

Active APIs

- POST `http://localhost:3000/api/indoor-analyze`
- POST `http://localhost:3000/api/indoor-photo-analyze`

Planned Backend APIs Referenced in Comments

- POST `/api/save-design`
- GET `/api/get-designs`
- GET `/api/get-design/:designId`
- DELETE `/api/delete-design/:designId`